Docker and secrets

Beyond build and run, and choosing how secrets are stored, delivered and revoked

21 October 2026

Contents

Scope and prerequisites **6**

1. Read Docker, Compose and secret tools **7**
1.1 Hello, world . . . 7
1.2 Image names . . . 7
1.3 Dockerfile instructions . . . 8
1.4 Running containers . . . 9
1.5 Compose files . . . 10
1.6 Reading output . . . 12
1.7 Look-alikes, and why both exist . . . 13
1.8 Where things live . . . 14
1.9 Encrypted files: SOPS and age . . . 15
1.10 Secret services: OpenBao and Vault . . . 16
1.11 Kubernetes manifests . . . 17
1.12 Everyday shorthand . . . 17

2. What an image and a container really are **19**
2.1 An image is layers plus configuration . . . 19
2.2 A container is a writable layer, namespaces and cgroups . . . 19
2.3 Seeing it: lab 1 . . . 20
2.4 Questions for an agent . . . 21
2.5 Exercise 1: predict what leaks . . . 22

3. Building beyond `docker build` . **22**
3.1 The `syntax` directive . . . 22
3.2 Multi-stage builds . . . 23
3.3 Cache mounts . . . 23
3.4 Build secrets and SSH forwarding . . . 23
3.5 Build checks . . . 24
3.6 Attestations, multi-platform builds and Bake . . . 25
3.7 Questions for an agent . . . 25
3.8 Exercise 2: review a Dockerfile . . . 25

4. Running beyond `docker run` **26**
4.1 Compose v5 . . . 26
4.2 Where values come from . . . 26
4.3 Start order, profiles and overrides . . . 27
4.4 Hardening a container . . . 27

4.5 Inspection as evidence . . . 28
4.6 Swarm, briefly . . . 29
4.7 Questions for an agent . . . 29
4.8 Exercise 3: read a Compose file . . . 29

5. Why the repository model breaks **30**
5.1 What git gets right, and what it assumes . . . 30
5.2 Access only grows until you rotate . . . 30
5.3 The secret lifecycle . . . 31
5.4 Secret zero . . . 31
5.5 Kinds of secret . . . 32
5.6 Questions for an agent . . . 32
5.7 Exercise 4: who holds the password? . . . 33

6. Secrets inside Docker **33**
6.1 Leak paths, collected . . . 33
6.2 Compose secrets without Swarm . . . 34
6.3 The `_FILE` convention . . . 36
6.4 Swarm secrets . . . 36
6.5 Rotation through a bind mount . . . 36
6.6 Registry credentials . . . 37
6.7 Questions for an agent . . . 38
6.8 Exercise 5: review a Compose change . . . 38
6.9 Exercise 6: predict a rotation . . . 38

7. Model one: encrypted files in git **39**
7.1 SOPS: envelope encryption per file . . . 39
7.2 age . . . 40
7.3 Lab 3: recipients, removal and history . . . 40
7.4 Getting values to the process . . . 41
7.5 KMS recipients change the model . . . 41
7.6 ansible-vault and git-crypt . . . 42
7.7 Strengths and weaknesses . . . 42
7.8 Questions for an agent . . . 43
7.9 Exercise 7: plan a departure . . . 43

8. Model two: a secret service **43**
8.1 The request path . . . 44
8.2 Lab 4: dynamic database credentials . . . 44
8.3 What it costs . . . 46

8.4 OpenBao or Vault 46
8.5 Hosted services 47
8.6 Questions for an agent 48
8.7 Exercise 8: design the bootstrap 49

9. Delivery, identity and comparison **49**
9.1 Three separate decisions 49
9.2 Delivery patterns for Compose hosts 49
9.3 Identity removes secrets 50
9.4 The models compared 51
9.5 Questions for an agent 52
9.6 Exercise 9: choose under constraints 53

10. Kubernetes later (optional) **53**
10.1 What a Kubernetes Secret is 53
10.2 Getting secrets from a store into pods 54
10.3 How today's choice carries forward 55
10.4 Questions for an agent 55
10.5 Exercise 10: predict an update 56

11. Choosing and migrating **56**
11.1 Questions that change the answer 56
11.2 Three candidate designs 56
11.3 A staged migration 57
11.4 Rotation without an outage 58
11.5 Acceptance criteria for a migrated service 58
11.6 Leak response 59
11.7 Diagnosis 59
11.8 Questions for an agent 61
11.9 Exercise 11: acceptance criteria 61
11.10 Exercise 12: diagnose 61

Worked answers **62**
Exercise 1 62
Exercise 2 62
Exercise 3 63
Exercise 4 63
Exercise 5 64
Exercise 6 64
Exercise 7 65

Exercise 8 65
Exercise 9 66
Exercise 10 67
Exercise 11 67
Exercise 12 68

Field reference **68**

Labs, sources and verification **70**
Files 70
Versions and sources 70
What was and was not verified 70

Scope and prerequisites

This tutorial opens with a reading primer in chapter 1, and then has two parts of unequal size. Part 1 (chapters 2–4) is a short tour of what Docker does beyond building, running and exec-ing into images, chosen for practical use and for the mechanics that later explain how secrets leak. Part 2 (chapters 5–11) is about secret management: why a private repository is the wrong model for secrets, what the current models are, where they differ, and how to choose and implement one for services run with Docker Compose on Linux hosts, with Kubernetes as a possible later platform.

Allow two to three hours. Read in order; chapter 10, on Kubernetes, is an optional branch that needs chapters 6 and 9. Most chapters end with an exercise; worked answers are collected near the end. Pause at each exercise before reading its answer.

Versions. Docker Engine 29.9 (29.9.2 was released on 20 October 2026), Docker Compose v5.6, Buildx 0.38, BuildKit 0.34 and Dockerfile frontend 1.28. For secrets: SOPS 3.13.3, age 1.3.2 and OpenBao 2.7.1, with PostgreSQL 18 as the service that consumes credentials. Kubernetes 1.37 is discussed but not run. HashiCorp Vault 2.2, cloud secret managers and hosted products are compared from their documentation. Several facts here changed in 2025–2026 (Compose jumped from v2 to v5; OpenBao stopped accepting audit devices through its API; Vault's licence and owner changed), so older material, and much of what a language model has learned from it, is stale.

Statements marked **observed** come from running the labs on 21 October 2026, on Docker Engine 29.9.1 with Compose 5.6.0 in a disposable Docker-in-Docker daemon; the final appendix and `verification.md` say exactly what ran. Documented behaviour is linked to its source. Recommendations are labelled as such. The `examples/` directory holds four labs; everything essential from them is reproduced here, and `README.md` explains how to run them.

1. Read Docker, Compose and secret tools

This chapter is a phrasebook: enough to recognise each part of a Dockerfile, a Compose file, a command's output and an encrypted secret file, and to know what to look up. Most items get a sentence and a small example, usually a fragment. Why things behave as they do comes in later chapters, so skim this now and return when a construct looks unfamiliar.

1.1 Hello, world

A **Dockerfile** is a recipe for an **image**. An image is a stored template and a **container** is a process started from one, so one image can back many containers. Saved as `Dockerfile`, this is a complete recipe:

```
FROM alpine:3.24
CMD ["echo", "hello"]
```

```
docker build -t hello .
docker run --rm hello    # prints: hello
```

`-t` names the image. The `.` is the **build context**, the directory whose files the build may copy. `.dockerignore` lists what to leave out of it. `--rm` deletes the container when it exits.

1.2 Image names

A name has the form `registry/namespace/repository:tag@digest`, and most parts can be omitted:

```
alpine                          docker.io/library/alpine:latest
alpine:3.24                     tag 3.24 of that repository
registry.example.com/api:1.4    another registry
alpine@sha256:be265606...       exact content
```

A **tag** is a movable label, and `latest` is only the default tag, not a promise of newness. A **digest** is the SHA-256 of the content, so it cannot move.

1.3 Dockerfile instructions

Each instruction is one line, by convention in capitals; \ continues a line and # starts a comment. Dockerfile reference.

Instruction	Meaning
`FROM image`	Start from this image; `FROM scratch` starts from nothing
`RUN cmd`	Run a command during the build and keep its filesystem changes
`COPY src dest`	Copy from the build context into the image
`WORKDIR /app`	Set the current directory for later instructions and at run time
`ENV KEY=value`	Set a variable during the build and in every container
`ARG KEY=default`	Set a variable during the build, given with `--build-arg`
`USER 1000`	Run later steps and the container as this user
`EXPOSE 80`	Document a port; it publishes nothing
`LABEL key=value`	Attach metadata to the image
`VOLUME /data`	Declare a path that holds data outside the container's layer
`HEALTHCHECK`	Give a command that reports whether the container is healthy
`ENTRYPOINT`, `CMD`	Set the command a container runs

Directives. A comment on the first lines of the file can instruct the parser. These two look like comments and are not; chapter 3 covers both:

```
# syntax=docker/dockerfile:1
# check=error=true
```

Stages. Each `FROM` starts a new **stage**, which `AS` names. A later stage can build on one or copy from it, and `--target` stops the build at one. Stages are the nearest thing to a unit of reuse; a Dockerfile has no func-

tions. A `--platform=` flag on `FROM` picks an architecture, and `$BUILD-PLATFORM` means the builder's own. This fragment shows only the stage syntax:

```
FROM golang:1.25 AS build
RUN go build -o /out/app ./cmd/app

FROM alpine:3.24
COPY --from=build /out/app /app
```

Mounts. `RUN --mount=type=…` gives one step something that does not become part of the image: `secret`, `ssh` or `cache`. Chapter 3 explains them.

Branches, loops and errors. A Dockerfile has no `if` or loop, so these are shell inside `RUN`. A `RUN` whose exit status is non-zero stops the build, and only the last command's status counts, so `RUN false; echo ok` succeeds and `RUN false && echo ok` fails. A here-document holds a longer script, and needs `set -e` to stop at its first failure:

```
RUN <<EOT
set -e
if [ "$VARIANT" = small ]; then echo small > size.txt; fi
EOT
```

1.4 Running containers

Command	Meaning
`docker run IMAGE [CMD]`	Create and start a new container
`docker exec CONTAINER CMD`	Run another process in a running container
`docker logs -f CONTAINER`	Follow what the main process has written to its output
`docker ps -a`	List containers, including stopped ones
`docker images`	List images
`docker inspect NAME`	Print full configuration and state as JSON

The flags you will see most on `docker run`:

```
docker run -d --name web -p 8080:80 -e MODE=prod \
  -v "$PWD/site:/usr/share/nginx/html:ro" nginx:1.29-alpine
docker exec -it web sh
```

`-d` runs in the background and `--name` sets the name. `-p` and `-v` read **outside first, inside second**: host port 8080 reaches container port 80, and the host's `site` directory appears read-only (`:ro`) at the inner path. `-e` sets an environment variable. `-it` attaches a terminal.

1.5 Compose files

A **Compose file**, normally `compose.yaml`, declares containers, networks and volumes that belong together. Each entry under `services:` is a kind of container. Compose file reference.

```
x-common: &common
  restart: unless-stopped

services:
  web:
    <<: *common
    image: nginx:1.29-alpine
    ports:
      - "8080:80"
    volumes:
      - ./site:/usr/share/nginx/html:ro
      - cache:/var/cache/nginx
    environment:
      GREETING: ${GREETING:-hello}
  probe:
    image: alpine:3.24
    command: ["wget", "-qO-", "http://web/"]
    depends_on: [web]
    profiles: [debug]

volumes:
  cache:
```

- **`x-` and `&`, `*`, `<<`.** A top-level key starting with `x-` is ignored by Compose (extensions). `&common` is a YAML anchor naming a value, `*common` refers to it, and `<<:` merges it into the map. This

is how a Compose file avoids repetition (fragments).

- **Names.** Containers find each other by service name, hence `http://web/`. The project is named after the directory, so in a directory called `shop` this makes the container `shop-web-1`, the network `shop_default` and the volume `shop_cache`.
- `environment:`. A map (`KEY: value`) and a list (`- KEY=value`) mean the same. A bare `- KEY` in the list passes through the value from the shell running Compose.
- `profiles:`. A plain `up` leaves the service out. Naming the profile (`--profile debug`) or the service (`docker compose run --rm probe`) includes it.
- `build:`. Used instead of or beside `image:`, this builds the image, and has its own settings: `context`, `dockerfile`, `target`, and `args` for values that reach `ARG`.
- `networks:`. Every service joins the project's default network unless it lists others, which the top-level `networks:` declares.
- `secrets:` **and** `configs:`. These appear at the top level and in a service, and chapter 6 covers secrets. `configs:` has the same shape for files that are not sensitive, mounted where the service asks.

Dollars. Compose replaces variables before anything runs:

Form	Value
`${VAR}`	The variable, or empty with a warning
`${VAR:-text}`	`text` if `VAR` is unset or empty
`${VAR-text}`	`text` only if `VAR` is unset
`${VAR:+text}`	`text` if `VAR` is set and not empty, otherwise empty
`${VAR:?message}`	Stop with `message` if `VAR` is unset or empty
`$$`	A literal `$`, so `$$5` reaches the container as `$5`

Commands. `docker compose` takes the same verbs as `docker`, applied to service names:

Command	Meaning
`docker compose up -d`	Create or update everything and start it
`docker compose ps`	List this project's containers
`docker compose exec web sh`	Run a process in the running `web` container
`docker compose run --rm web sh`	Start a new, one-off `web` container
`docker compose config`	Print the merged file with variables replaced
`docker compose down`	Remove containers and networks, keeping named volumes

1.6 Reading output

`docker ps` shows one line per container (here with only two of its columns):

```
STATUS       PORTS
Up 2 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

`0.0.0.0:8080->80/tcp` means that port 8080 on every IPv4 host address reaches port 80 in the container, and `[::]` is the same for IPv6. `80/tcp` with no arrow means the image declares the port and nothing is published. With a health check, the status can read `Up 2 hours (healthy)` or `(unhealthy)`. `Restarting (1)` means a restart policy is retrying a process that exited with code 1.

Build output. Lines such as `#6 [one 3/3] RUN echo hi` give a step number, the stage, and the instruction's place in that stage. `CACHED` means the step's earlier result was reused and the command did not run.

Exit codes. `Exited (N)` is the main process's exit code. Above 128 it means death by signal, with the signal number added: `Exited (137)` is 128 + 9, SIGKILL, which is what `docker kill` sends and what `docker stop` sends if the process has not exited after 10 seconds. Failing to start at all gives 127 from `docker run` when the command does not exist.

Double braces. `-f` or `--format` takes a Go template over the JSON, where `.State.Status` means that field and `json` prints a value as JSON:

```
docker inspect -f '{{ .State.Status }}' web
docker inspect -f '{{ json .Config.Env }}' web
```

Four kinds of substitution are easy to confuse. `${VAR}` in a Compose file is replaced by Compose on the machine running it. `$VAR` in a `RUN` line is expanded by the shell during the build. `${VAR}` in other Dockerfile instructions, such as `FROM` and `COPY`, is expanded by the Dockerfile parser. `{{ .Field }}` is a Go template, in `docker -f` and in OpenBao Agent's templates below.

1.7 Look-alikes, and why both exist

Exec form and shell form. `CMD ["sleep", "30"]`, with JSON brackets, runs the program directly. `CMD sleep 30` is stored as `/bin /sh -c "sleep 30"`, which gives shell features but can leave the shell, not your program, as process 1 to receive signals. `RUN`, `CMD` and `ENTRYPOINT` all accept both. Docker's reference recommends the exec form for `ENTRYPOINT`.

`ENTRYPOINT` and `CMD`. The container runs an exec-form `ENTRYPOINT` with `CMD` as its arguments; a shell-form `ENTRYPOINT` ignores `CMD`. With `ENTRYPOINT ["echo", "fixed"]` and `CMD ["default"]`, the container prints `fixed default`; `docker run IMAGE changed` replaces `CMD` and prints `fixed changed`, and only `--entrypoint` replaces the entrypoint. A `CMD` alone is the whole default command.

`ENV` and `ARG`. `ARG` exists for the build and `ENV` continues into every container, so `ENV` suits a default setting at run time. An `ARG` before the first `FROM` is usable only in `FROM` lines, and is empty inside the stage unless a bare `ARG NAME` there brings it in. Neither is a place for a secret, as chapter 2 shows.

`COPY` and `ADD`. `ADD` also fetches URLs and unpacks local tar files. `COPY` only copies, so it is the default choice.

`RUN` and `CMD`. `RUN` happens once at build time, and `CMD` every time a container starts.

`docker run`, `exec` and `start`. `run` creates a new container, `exec` adds a process to a running one, and `start` restarts a stopped one with its writable layer intact.

Stopping and removing. `docker stop` sends SIGTERM and later SIGKILL, and `docker kill` sends SIGKILL at once. Either way the container still exists until `docker rm`. In Compose, `stop` keeps containers, `down` removes them, and only `down -v` removes named volumes too.

Three kinds of mount. Storage.

```
volumes:
  - ./site:/srv/site     # bind mount
  - cache:/var/cache     # named volume
tmpfs:
  - /tmp/scratch         # memory
```

A **bind mount** is a host path, recognised by its leading `./` or `/`. A **named volume** is storage that Docker manages. A **tmpfs** is in memory and is gone when the container stops.

`docker compose` and `docker-compose`. With a space it is the current plugin. With a hyphen it is usually the obsolete Python program or a compatibility wrapper. `compose.yaml` is preferred to the older name `docker-compose.yml`; with both present, Compose warns and uses the first. `compose.override.yaml`, if present, is merged in automatically.

`.env` and `env_file:`. They look alike and do different jobs. Chapter 4 explains the difference, which is a frequent cause of secrets ending up in the wrong place.

1.8 Where things live

Path	What it is
`/var/lib/docker`	The daemon's data: images, container layers, volumes, logs
`/var/run/docker.sock`	The daemon's API; access to it amounts to root
`/etc/docker/daemon.json`	Daemon configuration

Path	What it is
`~/.docker/config.json`	Client settings and, often, registry credentials
`/run/secrets/NAME`	Where a secret appears inside a container, by convention
`compose.yaml`, `.env`	In the project directory

1.9 Encrypted files: SOPS and age

age is an encryption tool. Its public key, a **recipient**, starts `age1`; its private key, an **identity**, starts `AGE-SECRET-KEY-1` and is what must never leak. `age-keygen -o key.txt` creates a pair, and `age-keygen -y key.txt` prints the recipient.

SOPS encrypts the values in a structured file and leaves the keys readable. It finds the identity through `SOPS_AGE_KEY_FILE`, or by default in `~/.config/sops/age/keys.txt`. An encrypted YAML file looks like this, shortened:

```
db:
    password: ENC[AES256_GCM,data:fZnP,iv:...,tag:...,type:str]
replicas: ENC[AES256_GCM,data:Wg==,iv:...,tag:...,type:int]
sops:
    age:
        - enc: |
            -----BEGIN AGE ENCRYPTED FILE-----
            ...
          recipient: age16ljjqpuf...
    mac: ENC[AES256_GCM,data:I5fw...]
    version: 3.13.3
```

So `ENC[…]` is a ciphertext and the `sops:` block is metadata, listing who can decrypt. In a dotenv file the same metadata appears as lines such as `sops_mac=` and `sops_age__list_0__map_recipient=`. A file called `.sops.yaml` holds the rules for which recipients a new file gets, not secrets.

```
sops encrypt -i prod/app.yaml
sops decrypt --extract '["db"]["password"]' prod/app.yaml
```

Two others appear in chapter 7. A string starting `$ANSIBLE_VAULT;1`

`.1;AES256`, often after a `!vault |` tag, is ansible-vault ciphertext. A line such as `secretfile filter=git-crypt diff=git-crypt` in `.gitattributes` means git-crypt encrypts that path.

1.10 Secret services: OpenBao and Vault

OpenBao's command is `bao`. It was forked from HashiCorp Vault's `vault`, so instructions look alike and some names remain: the agent's configuration still has a `vault { … }` block. `BAO_ADDR` and `BAO_TOKEN` say where and as whom. A token starts `s.` in OpenBao and `hvs.` in recent Vault (commands).

Paths. Everything is addressed like a filesystem, and the leading components select what is mounted there, usually a **secrets engine**. In `database/creds/app`, `database/` is the engine, which treats `creds/app` as an instruction to issue a credential, so reading a path can create something. `auth/` holds auth methods, as in `auth/approle/login`, and `sys/` is the server's own API.

```
$ bao kv get secret/app/db
=== Secret Path ===
secret/data/app/db
...
====== Data ======
Key         Value
---         -----
password    LAB
```

A path that changes. As this output shows, the `kv` command turns `secret/app/db` into the API path `secret/data/app/db`, and a policy must name the API path. **Observed:** a token whose policy named `secret/app/*` got `permission denied` (KV v2).

HCL. Server, agent and policy files are in HashiCorp Configuration Language: `key = value`, and blocks with optional labels. A server file has blocks such as `storage`, `listener`, `seal` and `audit "file" "NAME"`. An agent file has `auto_auth`, which holds the login `method`, and `template`. A policy is a list of paths with their permissions (policies):

```
path "database/creds/app" {
  capabilities = ["read"]
```

```
}
```

`<<-EOT … EOT` is an HCL multi-line string. Agent templates use one to hold a Go template, in which `with secret` fetches a path and `.Data` is the reply:

```
template {
  destination = "/run/app/db.env"
  contents    = <<-EOT
    {{ with secret "database/creds/app" }}
    PGUSER={{ .Data.username }}
    {{ end }}
  EOT
}
```

Vocabulary. Chapter 8 defines each of these: **seal** and **unseal**, **lease** and **TTL** (time to live), **auth method**, **AppRole**, with its **role ID** and **secret ID**, and **dynamic credentials**.

1.11 Kubernetes manifests

Chapter 10 is optional and mentions a few Kubernetes objects. A manifest is YAML with `apiVersion`, `kind`, `metadata` and a body. In a Secret, `data:` values are base64, and `stringData:` takes plain text. A pod uses a Secret by name, as an environment variable through `secretKeyRef` or as files through a volume, and `subPath` mounts a single file from a volume. This excerpt follows the Kubernetes documentation and was not run:

```
apiVersion: v1
kind: Secret
metadata:
  name: api-creds
data:
  API_KEY: czNjcmV0
```

1.12 Everyday shorthand

Term	Meaning
Layer	One step's filesystem changes, stored in the image
Registry	A server that stores images, reached by `push` and `pull`
BuildKit, `buildx`	The current builder and its command-line plugin
Daemon, `dockerd`	The server that does the work; `docker` is its client
Context	The build's input directory, or (`docker context`) which daemon to use
Swarm	Docker's own clustering mode, separate from Compose
Secret zero	The first credential, which unlocks the others (chapter 5)

2. What an image and a container really are

Day-to-day use of `docker build` and `docker run` works with an image as an opaque artefact. Two facts about its structure explain most of part 2: an image is a stack of filesystem diffs plus a JSON configuration, and both are readable by anyone who has the image.

2.1 An image is layers plus configuration

An image is described by a **manifest** that lists a **configuration** blob and an ordered list of **layer** blobs, all addressed by their SHA-256 digest.

- Each **layer** is a tar archive of the filesystem changes made by one build step. A file deleted by a later step is not removed from the earlier layer; the later layer records a *whiteout* entry that hides it in the merged view (OverlayFS driver).
- The **configuration** holds the runtime defaults: `Env`, `Cmd`, `Entrypoint`, `User`, `WorkingDir`, labels, exposed ports, and a `history` array recording how each step was created.

Content addressing gives the integrity property that matters for supply chains: the same digest always means the same bytes. It gives no confidentiality at all. Anyone who can pull the image, or who can run `docker save` on a host that has it, holds every layer and the whole configuration.

Engine 29 uses the **containerd image store** by default on fresh installations; upgraded daemons keep the older overlay2 store until it is switched on, and the switch hides old images rather than deleting them (containerd image store). `docker info -f '{{ .DriverStatus }}'` shows `io.containerd.snapshotter.v1` when it is in use. The model above holds for both.

2.2 A container is a writable layer, namespaces and cgroups

`docker run` asks the daemon, `dockerd`, to create a container. containerd and `runc` build it from the image's layers plus a new **writable**

layer, place the process in fresh Linux namespaces (PID, mount, network, UTS, IPC, and optionally user) and cgroups, apply the image configuration, and then apply whatever the command line adds. The result is recorded as the container's own configuration, which `docker inspect` prints: environment variables, mounts, user, capabilities and the rest.

Three consequences matter later:

- What a process inside the container writes goes to the writable layer, on the host's disk under Docker's data root, unless it goes to a volume or a tmpfs. `docker commit` and `docker export` capture that layer.
- Environment variables given at run time are stored in the container's configuration and shown by `docker inspect`. Inside the container they are in `/proc/<pid>/environ` and are inherited by every child process.
- Whoever can talk to the Docker socket can run `docker inspect`, `docker exec` and `docker run -v /:/host`. Docker's own documentation says only trusted users should control the daemon, because that access is root on the host (daemon attack surface). Membership of the `docker` group is therefore a secret-access decision.

2.3 Seeing it: lab 1

`examples/leaks/Dockerfile.leaky` makes three common mistakes on purpose:

```
# syntax=docker/dockerfile:1
FROM alpine:3.24
ARG NPM_TOKEN
ENV DB_PASSWORD=LAB-DBPW-5e1c
COPY deploy_key /tmp/deploy_key
RUN test -n "$NPM_TOKEN" && wc -c /tmp/deploy_key
RUN rm /tmp/deploy_key
CMD ["sleep", "3600"]
```

The lab builds it with `--build-arg NPM_TOKEN`, then reads the image three ways. **Observed** on Engine 29.9.1:

```
history lines mentioning a marker:
RUN |1 NPM_TOKEN=LAB-NPM-7f3a /bin/sh -c rm /tmp/deploy_key #
 ↪  buildkit
RUN |1 NPM_TOKEN=LAB-NPM-7f3a /bin/sh -c test -n "$NPM_TOKEN" &&
 ↪  wc -c ...
ENV DB_PASSWORD=LAB-DBPW-5e1c
ARG NPM_TOKEN=LAB-NPM-7f3a
Config.Env: [..., "DB_PASSWORD=LAB-DBPW-5e1c"]
LAB-NPM-7f3a: FOUND in blobs/sha256/ea9d…  (the configuration
 ↪  blob)
LAB-DBPW-5e1c: FOUND in blobs/sha256/ea9d…
LAB-KEY-9b20: FOUND in blobs/sha256/6055…:tmp/deploy_key  (a
 ↪  layer)
```

Each line has a cause:

- A build argument used by `RUN` is recorded in that step's history entry, and an `ARG` stays in scope for every later `RUN` in the stage, so the token appears even on the `rm` line, which never used it. The Dockerfile reference warns against build arguments for secrets (ARG).
- `ENV` is part of the configuration by design: it is the default environment of every container started from the image (ENV).
- `rm` in a later step adds a whiteout; the key is still in the layer that `COPY` produced. `scan_image.py` found it by opening the layer tar from `docker save`.

The same image built from `Dockerfile.fixed` (chapter 3) showed none of the three markers in its history, configuration or layers.

At run time, `docker run -e DB_PASSWORD=…` put the value in three places (**observed**): `docker inspect`'s `Config.Env`, `/proc/1/environ` inside the container, and the environment of a child process started with `docker exec`.

2.4 Questions for an agent

Ask: "Show me every place this image or container stores the value of `X`." A good answer names the history, the configuration, specific layers, the container configuration and the process environment, and proves absence with commands such as `docker history --no-trunc`, `docker`

`image inspect` and a scan of `docker save` output. Warning signs: "the file is deleted in the Dockerfile, so it's gone", or "it is only a build argument".

2.5 Exercise 1: predict what leaks

For each Dockerfile fragment, say whether the secret value can be recovered from the final image, and from where.

1. `COPY id_ed25519 /root/.ssh/` then `RUN git clone … && rm -rf /root/.ssh` in one `RUN`, with the key copied in the previous step.
2. A multi-stage build: stage `build` has `ARG GITHUB_TOKEN` and a `RUN` that uses it; the final stage is `FROM alpine` with `COPY --from=build /out /app`.
3. `RUN --mount=type=secret,id=tok,env=TOKEN ./fetch-deps.sh`, where the script writes `~/.npmrc` containing the token.

Worked answer

3. Building beyond `docker build .`

BuildKit has been the default builder on Linux since Engine 23; the legacy builder is deprecated (deprecated features). It executes a build as a dependency graph rather than a sequence, which is what makes the features below possible. None of them is needed to build an image; each pays off under a specific pressure.

3.1 The `syntax` directive

```
# syntax=docker/dockerfile:1
```

The first line tells BuildKit to fetch the latest stable 1.x Dockerfile frontend (1.28.0 on 7 October 2026) rather than use the one built into the daemon. Newer options, such as `env=` on secret mounts (frontend 1.10), need it on older engines; a build that fails with "unexpected key 'env' " usually lacks it. The cost is a network fetch of the frontend image unless cached.

3.2 Multi-stage builds

```
FROM golang:1.25 AS build
WORKDIR /src
COPY . .
RUN go build -o /out/app ./cmd/app

FROM gcr.io/distroless/static
COPY --from=build /out/app /app
USER 65532
ENTRYPOINT ["/app"]
```

Only the final stage becomes the image. Compilers, source and intermediate layers of earlier stages are not in it, so the image is smaller and has less to patch. For secrets, this hides earlier stages' layers and history from the final image, but it is not a secret mechanism: build arguments are still recorded in detailed provenance (below) and the earlier stages persist in the build cache on the builder.

3.3 Cache mounts

```
RUN --mount=type=cache,target=/root/.cache/pip pip install -r
 ↪  requirements.txt
```

The cache directory persists across builds on that builder but is never part of a layer. Use it when dependency downloads dominate build time. It is a build-host optimisation; CI runners that start empty gain nothing unless the cache is exported.

3.4 Build secrets and SSH forwarding

The fixed Dockerfile from lab 1:

```
# syntax=docker/dockerfile:1
FROM alpine:3.24
RUN --mount=type=secret,id=npm_token,env=NPM_TOKEN,required=true
 ↪  \
    --mount=type=secret,id=deploy_key,required=true \
    test -n "$NPM_TOKEN" && wc -c /run/secrets/deploy_key
```

built with:

```
NPM_TOKEN=… docker build \
  --secret id=npm_token,env=NPM_TOKEN \
  --secret id=deploy_key,src=deploy_key .
```

A secret mount exists only for the duration of that one `RUN`: as a file (default `/run/secrets/<id>`, mode 0400) or, with `env=`, as an environment variable of that step only. `required=true` fails the build if the secret is not supplied, rather than silently building without it. On the CLI, `src=` reads a file, `env=` reads a variable, and `--secret id=X` alone reads the variable `X` (build secrets, RUN –mount).

Two caveats are easy to miss:

- **Cache.** "The contents of build secrets are not part of the build cache. Changing the value of a secret doesn't result in cache invalidation" (cache invalidation). A rotated token does not rerun the step that used it.
- **What the step writes persists.** The mount keeps the secret out of the layers; it cannot stop the command from copying it into the filesystem. This is not stated as a sentence in the documentation; it follows from the layer model in chapter 2. Package managers that write credentials into a config file (`.npmrc`, `pip .conf`, `.netrc`) are the usual cause.

For cloning private repositories, prefer `RUN --mount=type=ssh` with `docker build --ssh default`: the build talks to the host's SSH agent through a socket, and the private key never enters the build at all.

3.5 Build checks

`docker build --check .` evaluates lint rules without building and exits non-zero on a violation; `# check=error=true` at the top of a Dockerfile makes violations fail ordinary builds (build checks). On the leaky Dockerfile it reported two `SecretsUsedInArgOrEnv` warnings (**observed**), for `NPM_TOKEN` and `DB_PASSWORD`.

Know what that rule can see. It matches the **names** of `ARG` and `ENV` keys, never values, against a word list (`apikey`, `auth`, `credential(s)`, `key`, `password`, `pword`, `passwd`, `secret`, `token`) bounded by `_` or the ends of the name, with `public`, `file` and `version` exempt (rule; pat-

tern from BuildKit's `dockerfile2llb/validations.go`). `DB_PASSWORD` and `GITHUB_TOKEN` are flagged; `DBPASSWORD`, `MYTOKEN` and a secret pasted into a `RUN` command line or a `COPY`'d file are not. A clean check is weak evidence.

3.6 Attestations, multi-platform builds and Bake

- **Attestations.** BuildKit can attach SLSA provenance (minimal provenance is added by default) and an SBOM (`--sbom=true`) to an image. Provenance in `mode=max` "exposes the values of build arguments"; secret mounts "are never included" (provenance). Another reason not to pass secrets as build arguments.
- **Multi-platform.** `--platform linux/amd64,linux/arm64` builds one image index for several architectures, by emulation or with native builders.
- **Bake.** `docker buildx bake` builds several targets from an HCL, JSON or Compose file in parallel. Compose v5 builds through Bake.

3.7 Questions for an agent

Ask: "Which build steps need credentials, how does each one receive them, and what proves they are absent from the image?" A good answer uses secret or SSH mounts, notes any tool that writes credentials to disk and removes them in the same `RUN`, and offers a scan of the saved image as evidence. Warning signs: `ARG` for tokens "because it is only at build time"; relying on a clean `--check` as proof; a cache-busting argument missing where a rotated secret must take effect.

3.8 Exercise 2: review a Dockerfile

An agent submits:

```
FROM python:3.13-slim
ARG PIP_INDEX_URL
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . /app
ENV SENTRY_DSN=https://abc123@o1.ingest.sentry.io/42
USER 1000
```

```
CMD ["python", "/app/main.py"]
```

`PIP_INDEX_URL` is `https://user:token@pypi.internal/simple`. List the problems in order of severity, say which ones `--check` would catch, and sketch the fix.

Worked answer

4. Running beyond `docker run`

4.1 Compose v5

Compose is the declarative layer for "these containers, networks and volumes belong together". Its version history is a trap for both people and models: Compose v1 was the Python `docker-compose`; v2 the Go plugin `docker compose`; v5.0.0 (December 2025) followed v2.40.3, skipping 3 and 4 "to prevent (more) confusion with the obsolete docker-compose file versions 2.x and 3.x" (v5.0.0 notes). Docker describes v5 as functionally identical to v2 (history). The top-level `version:` key in a Compose file is obsolete and only produces a warning. An agent that adds `version: "3.8"` is drawing on old examples.

4.2 Where values come from

Two different mechanisms both look like "environment variables", and confusing them is a leading cause of secrets ending up in the wrong place.

Interpolation substitutes `${VAR}` in the Compose file itself before anything runs. Values come from the shell environment, then a file given with `--env-file`, then the project's `.env` (only if no `--env-file` was given) (interpolation). A variable in `.env` does not reach any container "by itself"; it reaches one only if `environment:` or `env_file:` references it.

Container environment is what the process sees. Highest precedence first: `docker compose run -e`; `environment` or `env_file` entries whose value is interpolated; `environment` literal values; `env_file`; the image's `ENV` (precedence).

`docker compose config` prints the fully interpolated model, so anything interpolated into `environment:` appears in plaintext in its output, and in any CI log that runs it. Lab 2 shows this (chapter 6).

4.3 Start order, profiles and overrides

```yaml
services:
  db:
    image: postgres:18-alpine
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 2s
      retries: 15
  app:
    depends_on:
      db:
        condition: service_healthy
```

`depends_on` alone orders container creation; with `condition: service_healthy` Compose waits for the dependency's health check to pass, and `docker compose up --wait` waits for the whole project. `profiles:` keeps services out of a default `up` until a profile is named (lab 4 starts its agent that way). Several `-f` files merge in order, which is the usual mechanism for per-environment overrides; `include:` instead loads another Compose file as its own sub-project with its own relative paths (include).

4.4 Hardening a container

Each of these reduces what a compromised process can do. All are long-standing `docker run` flags with Compose equivalents.

Setting	Effect	Cost
`user: "10001"`	Not root inside the container	Files and ports must suit that UID
`read_only: true`	Root filesystem read-only	Needs `tmpfs:` for scratch paths
`cap_drop: [ALL]`	Removes Linux capabilities	Add back only what is needed

Setting	Effect	Cost
`security_opt: [no-new-privileges: true]`	setuid binaries cannot gain privilege	Rarely any
`init: true`	A small init reaps zombies, forwards signals	None
`mem_limit`, `cpus`, `pids_limit`	Bounds resource use	Must be sized

Running as non-root matters directly for secrets: chapter 6 shows a non-root container failing to read a secret file that a root container reads without trouble.

Rootless mode runs the daemon and containers as an ordinary user; `userns-remap` maps container root to an unprivileged range but "the daemon itself is running with root privileges" (rootless). The containerd image store is not available with `userns-remap`.

4.5 Inspection as evidence

Question	Command
What did this container get?	`docker inspect` (`Config.Env`, `Mounts`, `HostConfig`)
What has it written?	`docker diff`
How was this image made?	`docker history --no-trunc`, `docker image inspect`
What happened, and when?	`docker events --since 1h`
What is using disk?	`docker system df -v`
Which daemon am I talking to?	`docker context ls`

Every one of these reads data that may contain secrets. That is the point of part 2: inspection is how you verify, and also how secrets leak.

4.6 Swarm, briefly

Swarm mode still ships in Engine 29 and SwarmKit is maintained; Docker makes no public statement about its future, and Mirantis has committed to supporting it through at least 2030 (Mirantis). It matters here only because Docker's best-designed secret mechanism is a Swarm feature (chapter 6). Adopting Swarm for that alone is rarely justified.

4.7 Questions for an agent

Ask: "For each service, list where every environment variable's value comes from, and run `docker compose config` to show me what is resolved." A good answer separates interpolation from container environment and points out values that will appear in `config` output. Warning signs: `version:` at the top; `.env` described as "passed to the containers"; hardening options added without checking that the image works as the configured user.

4.8 Exercise 3: read a Compose file

```
services:
  api:
    image: registry.example.com/api:1.4
    env_file: api.env
    environment:
      DATABASE_URL: postgres://api:${DB_PASSWORD}@db/api
      LOG_LEVEL: info
```

`.env` contains `DB_PASSWORD=s3cret` and `LOG_LEVEL=debug`; `api.env` contains `LOG_LEVEL=warning` and `DB_PASSWORD=other`. The image sets `ENV LOG_LEVEL=error`. What are `DATABASE_URL`, `LOG_LEVEL` and `DB_PASSWORD` inside the container, and which of these values appear in `docker compose config` output?

Worked answer

5. Why the repository model breaks

Keeping secrets in a private repository with tightly restricted access is a coherent model with a precise flaw. It is worth stating the model carefully, because each alternative in later chapters repairs a different part of it.

5.1 What git gets right, and what it assumes

For code, git provides history, review, attribution and reproducible deployment from a known commit. Access control lives on the remote: who may clone and push. The model assumes that **the value of a copy is bounded by the value of keeping it current**. A stale clone of code is harmless, and losing access to the remote is effectively losing access to the code.

A secret violates that assumption. A stale copy of a password is worth exactly as much as a fresh one until the password changes. Git is designed to replicate: every clone carries the whole history, and clones end up on laptops, in CI caches, in backups and in forks. Restricting the remote limits who can obtain a copy *from now on*; it says nothing about copies already made.

5.2 Access only grows until you rotate

One way to state it precisely: for a secret value s, let $A(s)$ be the set of principals (people, machines, backups, logs) that have ever been able to read it. Every copy, clone, log line and backup adds to $A(s)$; nothing removes from it. Removing a person from a repository, deleting the repository, or rewriting history does not shrink $A(s)$. The only operation that "revokes" is **rotation**: replacing s with a new value s', whose access set starts again from those who receive it.

Three consequences drive the rest of this tutorial:

1. **Revocation is rotation.** Any design question of the form "how do we revoke someone's access?" becomes "how cheaply can we rotate everything they could read?". A design that makes rotation cheap and routine is stronger than one that makes access control elaborate.

2. **Blast radius is about grouping.** A repository that holds all production secrets makes A the same for all of them; one leaked clone requires rotating everything. Separating secrets by consumer keeps access sets small and rotation local.
3. **You cannot observe $A(s)$ directly.** You can record who *was given* access and, with a service, who *fetched* a value. Neither tells you who copied it afterwards. Audit narrows uncertainty; it does not remove it.

5.3 The secret lifecycle

Each model is a set of answers to these stages:

Stage	Question
Create	Who generates the value, with what entropy, and does anyone see it?
Store	Where is the source of truth, and how is it protected at rest?
Distribute	How does the value reach the consuming process?
Use	Where does it live while in use: file, environment, memory?
Rotate	How is it replaced, and how do consumers pick up the new value?
Revoke	How is an old value made useless?
Audit	Who was given it, who fetched it, when?

The repository model answers "store" and "distribute" with git and leaves the rest to people. That is why it does not scale: the work of rotation, and the knowledge of who holds what, grow with the number of secrets and consumers, while git's automation helps only with copying.

5.4 Secret zero

Every scheme reduces many secrets to fewer, better-protected ones: a SOPS file is protected by an age key, a secret service by an authentica-

tion credential, a cloud secret manager by an IAM identity. Something must authenticate the very first request. This bootstrap credential is called **secret zero**, and asking "what is secret zero, who holds it, and how is it rotated?" is the fastest way to evaluate any proposal. Good answers tie it to something already established: a human's single sign-on, a host identity created at provisioning, a cloud instance identity, a CI provider's signed token. Weak answers move a long-lived password one step further away and stop.

5.5 Kinds of secret

Rotation cost varies by kind, and that cost shapes the design:

- **Credentials you issue**, such as database passwords and internal API tokens. You control both sides, so they can be rotated automatically or replaced by short-lived, per-consumer credentials (chapter 8).
- **Credentials issued to you**, such as third-party API keys and registry tokens. Rotation means a vendor console or API, and often a manual step.
- **Keys**, for signing, encryption, TLS and SSH. Rotating them affects everyone who verifies or decrypts, and losing an encryption key can destroy data.
- **Root and recovery material**: age identities that decrypt everything, OpenBao unseal or recovery keys, cloud root accounts. Rarely used and disastrous to lose either way; they need an offline, split, tested recovery procedure rather than automation.

5.6 Questions for an agent

Ask: "For this secret, list every principal that can read it today, including through history, backups, CI and images, and what it would take to rotate it." A good answer enumerates copies, not just permissions, and estimates rotation effort per consumer. Warning signs: "only three people have access to the repo" offered as the whole answer; "we'll purge it from git history" offered as revocation.

5.7 Exercise 4: who holds the password?

The production database password has been in a private repository since 2022. Five people have had access; two have left, and their access was removed. A CI job clones the repository to deploy, and the runner caches workspaces. The password is passed to the application with `docker run -e`. Nightly host backups include `/var/lib/docker`. Enumerate the access set as fully as you can, then say what must happen for the two leavers to lose access.

Worked answer

6. Secrets inside Docker

Whatever stores the secret, it must eventually reach a process in a container. This chapter covers what Docker itself offers for that last step, and how each option behaves under inspection and rotation.

6.1 Leak paths, collected

Mechanism	Where the value ends up	Who can read it
`ARG` used by `RUN`	Image history, max-mode provenance	Anyone with the image
`ENV` in a Dockerfile	Image configuration	Anyone with the image
`COPY` then `rm` later	An earlier layer	Anyone with the image
`docker run -e`, `--env-file`	Container configuration; process environment	Docker socket users; the container's processes and children
Compose `environment:`, `env_file:`	As above, plus `docker compose config` output	As above, plus CI logs
Value printed by the app	Container logs, log shipping	Whoever reads logs

Docker's Compose documentation is blunt: "Don't use environment

variables to pass sensitive information, such as passwords, in to your containers. Use secrets instead" (environment variables). Environment variables are inherited by every child process, appear in `/proc/<pid>/environ` to anyone with the same UID or ptrace rights (proc_pid_environ(5)), and are routinely dumped by error reporters and debug endpoints. Files can be given a mode and owner, are not inherited, and are not shown by `docker inspect`.

That is a preference, not an absolute. Many programs accept credentials only through the environment. When that is the case, set the variable in the process that needs it, at the last moment (a wrapper reading a file, or `sops exec-env`, chapter 7), rather than in the container's configuration.

6.2 Compose secrets without Swarm

A top-level `secrets:` element declares secrets; a service lists those it receives; each appears in the container under `/run/secrets/<name>` (use secrets). Outside Swarm, Compose supports two sources, and they work differently:

- `file:` bind-mounts a single host file read-only. `uid`, `gid` and `mode` in the service's long syntax "are silently ignored" for this source (services: secrets); the host file's own owner and mode apply.
- `environment:` takes a variable from Compose's own environment and *copies* it into the container's writable layer through the copy-to-container API (Compose source, `pkg/compose/secrets.go`). It honours `uid`, `gid` and `mode`, and is refused for services with `read_only: true`.

`external:` and the `driver` options are Swarm-only; outside Swarm Compose errors on them. Neither source encrypts anything.

Lab 2 (`examples/compose/`) runs PostgreSQL and a client, `app`, that runs as UID 70:

```
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
```

```
    secrets: [db_password]
  app:
    image: postgres:18-alpine   # used only for its psql client
    user: "70:70"
    environment:
      LEGACY_API_KEY: ${LEGACY_API_KEY}   # anti-pattern, for
↪   comparison
    secrets:
      - db_password
      - source: api_key
        uid: "70"
        gid: "70"
        mode: 0400
secrets:
  db_password:
    file: ./secrets/db_password
  api_key:
    environment: API_KEY
```

Observed, step by step:

1. `docker compose config` printed `LEGACY_API_KEY: LAB-LEGACY-11aa` but, for the two secrets, only the file path and the variable name.
2. With the host file at mode 0600 owned by the user running Compose, `db` started and initialised normally, but `app` got `Permission denied` on `/run/secrets/db_password` and could not log in. The official PostgreSQL entrypoint reads `*_FILE` while still root and then drops to the `postgres` user (entrypoint), so the root-only file worked for `db` and not for `app`, which is non-root from its first instruction.
3. With the file at 0444 inside a 0700 directory, both worked. The directory keeps other host users out; the container sees only the mounted file, so the directory's mode does not affect it.
4. `docker inspect` showed `bind …/secrets/db_password -> /run/secrets/db_password rw=false`. `api_key` was `-r-------- 70 70`, as requested. A `docker commit` of the running `app` produced an image containing `api_key` with its value, and an empty `db_password` mount point.
5. Compose printed `uid`, `gid` and `mode` "not supported outside

Swarm mode and will be ignored" warnings, yet applied all three to the environment-sourced secret. The documentation, not the warning, describes the behaviour.

So the file source keeps secrets out of the container's layer but leaves ownership to the host; the environment source lets you set ownership but writes the value to disk inside the container, and requires the value in the environment of whoever runs `docker compose`.

6.3 The `_FILE` convention

`POSTGRES_PASSWORD_FILE` is not a Docker feature. It is a convention of the Docker Official Images' entrypoint scripts, supported by PostgreSQL, MySQL, MariaDB and a number of others; each image documents which variables accept it (postgres, "Docker Secrets"). How well an image handles it varies. PostgreSQL's entrypoint reads the file, *exports* the value as `POSTGRES_PASSWORD` for initialisation, and then unsets every `POSTGRES_*` variable before starting the server; a less careful entrypoint might leave the value in the server's environment. For your own images, read secrets from files in the application rather than converting them back to environment variables in a wrapper.

6.4 Swarm secrets

Swarm's design is the one the Compose syntax imitates. A secret is sent to a manager over mutual TLS, stored in the Raft log (encrypted at rest), and delivered only to nodes running a task that needs it, decrypted into a tmpfs at `/run/secrets/<name>` on Linux. Secrets are immutable and limited to 500 KB; rotation is: create a new secret, update the service to remove the old one and add the new one under the same target, remove the old one. With `--autolock`, the managers' Raft encryption keys are themselves encrypted and a restarted manager needs a human to run `docker swarm unlock` (swarm secrets, autolock). It works only for Swarm services, not for standalone containers or Compose without Swarm.

6.5 Rotation through a bind mount

Lab 2 continues: the host file is replaced atomically (`printf … > db_password.new; mv db_password.new db_password`). **Observed:**

```
app sees: LAB-DBPW-v1
after 'up -d': LAB-DBPW-v1
after 'restart app': LAB-DBPW-v2
app login: FAILED
after recreating both: db container reads LAB-DBPW-v2
app login: FAILED
ALTER ROLE
app login: ok
```

Three distinct lessons:

- **A single-file bind mount pins an inode.** `mv` points the host's directory entry at a new inode; the container still holds the old one. An in-place write (`>` on the existing file) would be visible, but would also expose a partly written file to a reader. Mounting a **directory** instead makes renames visible (lab 4 does this). `docker compose up -d` did not recreate `app`, because nothing in its configuration changed; `restart` re-resolved the mount.
- **Delivering a secret is not rotating it.** PostgreSQL reads `POSTGRES_PASSWORD_FILE` only when initialising an empty data directory. The new file reached both containers, and the database still expected the old password. Rotation needs the authority (here the database) to accept the new value, usually while still accepting the old one for a transition.
- **Consumers must reload.** Even when the file changes visibly, a process that read it at start-up keeps the old value until it rereads or restarts.

6.6 Registry credentials

`docker login` without a credential helper stores credentials in `~/.docker/config.json` "in a base64-encoded format", which the documentation calls less secure than a credential store; `credsStore` and `credHelpers` delegate to `pass`, the Secret Service or the OS keychain, and `--password-stdin` keeps the password out of shell history (docker login). `config.json` on build hosts and CI runners belongs in the inventory of secrets.

6.7 Questions for an agent

Ask: "Show how each container receives each secret, which UID reads it, what happens when the secret changes, and how the authority that checks it learns the new value." A good answer distinguishes file and environment sources, checks ownership against the container's user, and describes a rotation sequence that includes the authority. Warning signs: secrets with `uid`/`mode` on a `file:` source presented as protection; `external: true` outside Swarm; rotation described as "update the file and run `up -d`".

6.8 Exercise 5: review a Compose change

An agent proposes this to remove passwords from `environment:`:

```
services:
  worker:
    image: registry.example.com/worker:2.1    # runs as UID 1000
    secrets:
      - source: smtp_password
        mode: 0400
        uid: "1000"
secrets:
  smtp_password:
    file: /etc/myapp/smtp_password    # written by Ansible, owner
↪   root, mode 0600
```

The agent reports: "Tested with `docker compose config`: valid." Predict what happens at run time, explain why the test did not catch it, and give two acceptable fixes with their tradeoffs.

Worked answer

6.9 Exercise 6: predict a rotation

Lab 4 renders credentials into `/run/app/db.env` on a volume mounted at `/run/app` (a directory). The agent rewrites the file by rename every time the credential changes. The application reads `db.env` once at start-up and keeps a connection pool. What does the application do when the credential is replaced and the old one revoked, and what would fix it?

7. Model one: encrypted files in git

The smallest change to the repository model keeps git as the store and distribution channel but stores only ciphertext. Git then provides history, review and deployment from a commit; cryptography decides who can read. The flaw from chapter 5 remains in a sharper form: whoever could decrypt a commit can decrypt it for ever.

7.1 SOPS: envelope encryption per file

SOPS (a CNCF Sandbox project, MPL-2.0; 3.13.3 released 23 July 2026) encrypts structured files so that **keys stay readable and values do not** (security model):

- For each file it generates a random **data key** and encrypts every value with AES-256-GCM under it.
- It encrypts the data key separately to each **master key** (a recipient): age, PGP, AWS KMS, GCP KMS, Azure Key Vault, HuaweiCloud KMS, or the transit engine of HashiCorp Vault or OpenBao. These wrapped copies live in the file's `sops` metadata.
- A message authentication code over the values detects tampering with the file as a whole, such as an entry removed (common operations).

This is **envelope encryption**, the same design that cloud KMS services use: a cheap symmetric key per object, wrapped by a small number of well-protected keys. Anyone holding any one recipient's key can unwrap the data key and read every value.

A `.sops.yaml` file chooses recipients by path:

```
creation_rules:
  - path_regex: prod/.*
    age: age1alice…,age1web1…
  - path_regex: dev/.*
    age: age1alice…,age1ci…
```

Key groups with `shamir_threshold` can require keys from several groups (for example, two of three administrators) to decrypt (key

groups).

7.2 age

age (1.3.2, BSD-3-Clause) is a small file-encryption tool and format. A recipient is an X25519 public key (`age1…`) or an SSH public key; an identity is the matching private key. Version 1.3.0 added hybrid post-quantum recipients (`age1pq1…`, ML-KEM-768 with X25519) (age 1.3.0). Whether SOPS accepts post-quantum recipients was not verified. With age, the recipient list in `.sops.yaml` *is* the access-control list, and the identity files are the secret zero.

7.3 Lab 3: recipients, removal and history

`examples/sops/lab.sh` creates three identities, `alice` (a person), `ci` and `web1` (a production host), and a git repository using the rules above. **Observed** with SOPS 3.13.3 and age 1.3.2:

```
=== 2. Encrypt a production env file
DB_USER=ENC[AES256_GCM,data:O3Pf,iv:86+TFZ9l/xtDdjVPY/02JvKc…
DB_PASSWORD=ENC[AES256_GCM,data:xukUEef/fKcyGshSWA==,iv:2orx…
=== 3. Who can decrypt
web1: can decrypt prod
ci: cannot decrypt prod
=== 4. Alice leaves: remove her as a recipient and commit
alice: cannot decrypt HEAD
alice reads HEAD~1 (from history): DB_PASSWORD=LAB-PROD-PW-1
=== 5. Rotate the data key too; history is unchanged
alice: cannot decrypt HEAD after rotate
alice reads HEAD~2 (from history): DB_PASSWORD=LAB-PROD-PW-1
=== 6. Tampering is detected
… Could not decrypt with AES_GCM: cipher: message authentication
 ↪  failed
```

In dotenv format, every value is encrypted, including the harmless `DB_USER`. In YAML or JSON, `encrypted_regex` or `unencrypted_suffix` can leave chosen values readable.

Step 4 is chapter 5's argument made concrete. `sops updatekeys` rewraps the existing data key for the new recipient list, so the new commit is closed to Alice; the old commit still carries a copy of the data

key wrapped to her. `sops rotate` generates a new data key, but the *values* are unchanged and the old commits are untouched. The SOPS documentation's procedure after a key is compromised is `updatekeys`, then `rotate`, then rotate the real credentials (key management). Only the last step revokes anything.

Step 6 renamed a key (`DB_USER` to `DB_ADMIN`) in the ciphertext file; the value's authentication failed, because SOPS passes each value's key path to AES-GCM as additional authenticated data (`Tree.Encrypt` in `sops.go`; the documentation does not describe this). Integrity is a real strength of the model: a commit cannot silently swap values around.

7.4 Getting values to the process

- `sops exec-env FILE CMD` decrypts into the environment of one child process (step 7 printed the length of `DB_PASSWORD` from inside it); `sops exec-file` passes a temporary FIFO or file path. Both avoid a plaintext file at rest (advanced usage).
- `sops decrypt --extract '["DB_PASSWORD"]'` writes a single value, for example to a file that Compose bind-mounts as a secret. Step 7 wrote it with `umask 077` and showed `git status` reporting it as an untracked file: a decrypted file inside the working tree is one `git add -A` away from being committed. Decrypt outside the repository, or list such paths in `.gitignore`.
- In configuration management, decryption usually happens at deploy time: Ansible's `community.sops` collection reads SOPS files as variables, and Flux decrypts them natively in Kubernetes (community.sops, Flux and SOPS).

7.5 KMS recipients change the model

If the recipient is a cloud KMS key (or OpenBao transit) rather than an age identity, each decryption is an authenticated API call. That changes three properties: the KMS can log every decryption (audit of use, not just of grants), access can be withdrawn centrally (future decryptions fail, though plaintexts already obtained are not recalled), and there is no private key file to leak. The price is a dependency on the KMS being reachable at decryption time, and secret zero becomes the identity used to call it. This is a step towards model two while keeping git as the store.

7.6 ansible-vault and git-crypt

ansible-vault encrypts whole files or single values (`!vault` tags from `encrypt_string`) with AES-256 in CTR mode and an HMAC-SHA256, using keys derived from a *password* with PBKDF2-SHA256 (10,000 iterations) (vault format). Everyone who decrypts shares that password, so there is no per-recipient access at all; removing one person means a new password and re-encrypting everything, and encrypted variables cannot be re-keyed in place. Vault IDs let different files use different passwords. It is adequate inside an existing Ansible repository with a small, stable group; it is the weakest option here on revocation.

git-crypt (0.8.0, September 2025; its README says it "has not yet reached maturity") encrypts whole files transparently through git filters, using deterministic AES-256-CTR. Its README is explicit that equality and length of files leak, that file names and commit messages are not encrypted, and that access cannot be revoked: there is no command to remove a GPG user, and "a user having the previous key can still access previous repository history" (git-crypt). Not recommended for new use.

7.7 Strengths and weaknesses

Property	Encrypted files in git
Source of truth	The repository; reviewable diffs of *which* keys changed
Infrastructure	None beyond git and key files (or a KMS)
Availability	Decryption works offline (age); no service to depend on
Access control	Recipient lists per path; coarse
Audit of use	None with age; per-decryption with a KMS recipient
Revocation	Only by rotating every value the person could read
Rotation	Manual: edit, commit, deploy, and change the authority
Dynamic credentials	No

Property	Encrypted files in git
Secret zero	Age identity per person and per host, or KMS identity

The model fits a small team, a few dozen hosts, mostly static secrets, and an existing git and configuration-management workflow. It degrades as the number of people and recipients grows, because each departure means a rotation campaign, and it gives no record of who actually read what.

7.8 Questions for an agent

Ask: "For this `.sops.yaml`, who can decrypt each file, where do their identities live, and what is our procedure when someone leaves?" A good answer maps path rules to recipients, names the storage of each identity, and includes rotating the credentials themselves. Warning signs: "run `sops updatekeys`" as the whole leaver procedure; an age identity committed alongside the repository or baked into an image; plaintext output written inside the working tree.

7.9 Exercise 7: plan a departure

The repository uses the `.sops.yaml` above. Alice leaves today. She had `prod/` and `dev/` access for two years; her laptop is returned and wiped. Write the procedure, in order, with what each step achieves. Then say what would be different if the recipients were a cloud KMS key that Alice could use only through her SSO identity.

Worked answer

8. Model two: a secret service

The second model replaces a file that is copied with a service that is asked. The value never has to exist anywhere but the service and the consumer; each request is authenticated, authorised and logged; and the service can issue credentials that it creates, limits in time and revokes.

8.1 The request path

```
consumer --(1) authenticate with an identity--> auth method -->
 ↪  token
consumer --(2) read path P with token--> policy check --> secret
 ↪  + lease
         <-(3) renew the lease, or fetch again before expiry
service  --(4) audit log: who, which path, when (values HMAC'd)
```

- **Authentication methods** turn some existing identity into a short-lived token: AppRole (a role ID plus a secret ID), JWT/OIDC (a person's SSO, a CI provider's signed token), Kubernetes service-account tokens, TLS client certificates, cloud instance identities (AppRole, JWT/OIDC).
- **Policies** grant capabilities (`read`, `create`, `update`, `list`, …) on path patterns. Deny is the default.
- **Secrets engines** serve values. The key–value engine (KV v2) stores versioned static secrets, keeping 10 versions by default with soft delete and destroy. The **database** engine creates a new database user for every read and drops it when the lease ends. The **transit** engine encrypts and decrypts for clients without handing out its keys; SOPS can use it as a recipient.
- **Leases** attach a time-to-live to issued credentials. They can be renewed up to a maximum and revoked individually or by path prefix (leases).
- **Audit devices** log every request and response, with secret values replaced by HMACs so that the log itself is not a secret store.
- **Seal.** The service's storage is encrypted; at start-up it is *sealed* and cannot decrypt anything until it is unsealed, either by a threshold of Shamir key shares held by people or automatically by a KMS or HSM (seal).

8.2 Lab 4: dynamic database credentials

`examples/openbao/` runs PostgreSQL, an OpenBao 2.7.1 server in **dev mode** (in memory, unsealed, with a fixed root token: a lab shortcut, never a deployment), an OpenBao Agent and a client. **Observed**:

1. **Root rotation.** After OpenBao was configured with the PostgreSQL admin password, `database/rotate-root` changed it.

Logging in with the original password was then rejected: only OpenBao knows the admin credential.

2. **One user per read.** Two reads of `database/creds/app` produced two distinct users, `v-token-app-…`, each created with a `VALID UNTIL` 40 seconds ahead (`default_ttl=40s`).
3. **Policy.** An AppRole token with the `app` policy could read `database/creds/app` and got `permission denied` reading the engine's configuration.
4. **Agent.** The agent, authenticated with AppRole, rendered `/run/app/db.env` (`-r-------- 70 70`) onto a memory-backed volume shared with the client, which connected as `v-approle-app-…`.
5. **Revocation.** `bao lease revoke -prefix database/creds/app` dropped every dynamic role ("dynamic roles left: 0"). The client's next login failed with `password authentication failed`.
6. **Recovery.** About 30 seconds later the agent had rendered a new credential and the client connected as a new user. The agent renews a renewable lease after two thirds of its duration (agent templates); with the lease revoked, renewal failed and the agent fetched a new credential.
7. **Audit.** The response entry for one issue recorded the path, the auth method (`display_name: approle`) and the client's address, with `"username":"hmac-sha256:…"` and `"password":"hmac-sha256:…"`.

One further check, run separately against the same stack, matters for incident response: a session opened with a dynamic credential **kept working after its lease was revoked and its role dropped**; a query sent afterwards on the open connection succeeded. Revocation and `VALID UNTIL` stop new logins. Ending existing sessions needs revocation statements that terminate them, or action at the database.

What the lab gives that model one cannot: each consumer has its own credential, it expires without anyone remembering to rotate it, the admin password is held by no person, a single command invalidates every issued credential, and every issue is logged. The log identifies a consumer only as precisely as its identity: with one shared AppRole, as in the lab, entries show the auth method and address, not which service asked. Per-host or per-service roles (whose names and entity IDs ap-

pear in the entry's `auth` metadata) make the record attributable; the lab did not exercise this.

8.3 What it costs

- **A tier-0 dependency.** Consumers that fetch at start-up cannot start while the service is down or sealed. Agents with caching soften this for static secrets; dynamic credentials cannot be issued without the service. Production means a highly available cluster (OpenBao's integrated Raft storage, three or five nodes), backups (snapshots), monitoring, upgrades and a tested unseal procedure.
- **Secret zero is still there.** Lab 4 placed the AppRole role ID and secret ID on disk, world-readable inside a 0700 directory, so the non-root agent could read them through a directory mount; a directory mount, unlike the single-file mount in chapter 6, enforces the directory's own mode inside the container (the agent first failed with "no known role ID" when the directory was 0700 and owned by root). In production the secret ID is a provisioning-time credential: delivered once by Ansible, limited in uses or lifetime, bound to source addresses, or delivered as a *response-wrapped* single-use token (response wrapping). Better still is an identity the platform already provides (chapter 9).
- **Unseal and recovery material** become the most sensitive secrets in the organisation: Shamir shares held by named people, or the KMS key used for auto-unseal.
- **Applications change**: either they learn to re-read files and reconnect, or an agent restarts them; dynamic credentials need applications that tolerate the user name changing.

8.4 OpenBao or Vault

HashiCorp Vault introduced the model. A licence change announced in August 2023, first applying to 1.15, put it under the Business Source Licence 1.1: production use is permitted unless you offer a competing hosted or embedded product, and each version converts to MPL-2.0 four years after release (licence). IBM completed its acquisition of HashiCorp on 27 February 2025. Vault moved to 2.0.0 in April 2026 and 2.2.0 is current. HashiCorp's hosted *HCP Vault Secrets* reached end of sale on 30 June 2025, with deletion of applications after end of

life (IBM notice), a reminder that a hosted product's lifetime is part of its risk.

OpenBao forked from Vault 1.14 (the last MPL release) under the Linux Foundation, is an OpenSSF Sandbox project, and remains MPL-2.0 (what is OpenBao). It is API compatible at the level used here, added namespaces in 2.3, and has diverged elsewhere: since 2.3.2 (August 2025, a security release) audit devices cannot be created through the API unless the server allows it, and must be declared in configuration. The lab's first attempt, `bao audit enable file …`, failed with "cannot enable audit device via API; use declarative, config-based audit device management instead"; the working lab declares it in `server .hcl`. Instructions written for Vault therefore need checking against OpenBao. Migration from Vault is tested only from 1.14.1 (migration).

For a new self-hosted deployment without Vault Enterprise features, OpenBao avoids licence and vendor questions at the cost of a smaller ecosystem (**recommendation**). Choosing Vault is reasonable where a support contract or Enterprise features (for example, performance replication) are wanted.

8.5 Hosted services

Service	Model	Notes
AWS Secrets Manager	Cloud, IAM auth	$0.40 per secret-month + $0.05 per 10,000 calls; rotation by Lambda; staging labels, not linear history (pricing)
GCP Secret Manager	Cloud, IAM auth	$0.06 per active version per location-month; $0.03 per 10,000 accesses (pricing)

Service	Model	Notes
Azure Key Vault	Cloud, Entra ID auth	$0.03 per 10,000 operations; no per-secret charge (Standard) (pricing)
Infisical	SaaS or self-hosted	MIT core, commercial `ee/` features (licence)
Doppler	SaaS	Proprietary; injects as environment via `doppler run`; on-premises only on Enterprise
1Password	SaaS, optional Connect server	Service accounts are rate-limited; Connect is self-hosted and caches (docs)
Bitwarden Secrets Manager	SaaS; self-host on Enterprise	Bitwarden Licence for commercial parts

Prices are list prices read on 21 October 2026. A KMS is not a secret manager: a KMS holds keys and wraps data keys (as a SOPS recipient); a secret manager stores, versions and serves values.

Cloud managers are excellent for workloads inside that cloud, where the platform supplies an identity for free. From hosts outside it, every host needs a cloud credential, and that credential becomes secret zero, unless the cloud's workload identity federation accepts an identity the hosts already have. SaaS products trade operations for a dependency on a vendor's availability, pricing and continued existence.

8.6 Questions for an agent

Ask: "If the secret service is unavailable for an hour, what fails, in what order, and what keeps working?" A good answer distinguishes running processes from restarts, cached static secrets from dynamic ones, and lease expiry from service outage. Warning signs: no answer about unseal; dev mode or a root token anywhere outside a lab; "high availability" without saying what holds the unseal keys.

Ask: "After we revoke a credential, how do we know it can no longer be used?" A good answer covers new logins *and* existing sessions, and names the evidence (the database's session list, the authority's own logs). Warning sign: "the lease is revoked" offered as proof.

8.7 Exercise 8: design the bootstrap

Thirty hosts run Compose services that will fetch credentials from an OpenBao cluster through an agent on each host, using AppRole. Hosts are built by OpenTofu and configured by Ansible. Propose how each host's role ID and secret ID are created, delivered, stored and replaced, and state what an attacker who obtains one host's disk image could do.

Worked answer

9. Delivery, identity and comparison

Chapters 7 and 8 can be compared fairly only after splitting "secret management" into three decisions that proposals often blur.

9.1 Three separate decisions

1. **Storage: where is the source of truth?** Encrypted files in git; a secret service; a cloud manager; a platform object (Kubernetes Secret, Swarm secret).
2. **Delivery: how does the value reach the process?** A file written by configuration management; a file rendered by an agent; an environment variable set by a wrapper; a direct API call from the application; a platform mount.
3. **Identity: how does the consumer prove who it is?** Possession of a key file; a password or AppRole secret ID; a signed token issued by a platform (CI, Kubernetes, cloud); a certificate.

The choices are largely independent. SOPS storage can be delivered by Ansible files or by Flux in Kubernetes; OpenBao storage can be delivered by an agent, the application itself, or a Kubernetes operator. Identity is what decides how much is left for secret zero.

9.2 Delivery patterns for Compose hosts

Pattern	How	Good for	Watch for
A. CM writes files	Ansible decrypts SOPS or vault data, writes `/etc/<svc>/secrets/*` (root, 0700 dir), Compose `file:` secrets, handler restarts service	Static secrets; no new service	Plaintext at rest on host; restart on change; ownership versus container UID
B. Host agent	OpenBao Agent under systemd renders files into a root-owned directory; `exec` restarts the service on change	Central store, dynamic credentials, existing Compose files	Agent's own credential; directory not file mounts; restart storms
C. Sidecar agent	Agent container renders into a shared tmpfs volume (lab 4)	Per-project isolation; nothing on host disk	One identity per project; application must re-read
D. Application fetches	Application uses the API or SDK at start and on expiry	Dynamic credentials with in-process reconnection	Code changes; every language needs a client

The OpenBao Agent's template `exec` block runs a command when a rendered file's content changes; the older `command` option is deprecated (agent templates).

9.3 Identity removes secrets

Where a platform already authenticates a workload, a long-lived secret can be replaced by an exchange of short-lived, signed tokens:

- **CI.** GitHub Actions can issue an OIDC token per job, which a

cloud provider or OpenBao's JWT auth method can exchange for short-lived credentials limited to that repository, branch or environment. No deployment password is stored in CI (GitHub OIDC).

- **Kubernetes.** Service-account tokens authenticate pods to OpenBao's Kubernetes auth method or to cloud workload identity.
- **Clouds.** Instance and workload identities authenticate to that cloud's KMS and secret manager.
- **SPIFFE/SPIRE** (CNCF graduated) issues short-lived workload identities across platforms, including bare metal, attested from properties of the node and process.
- **People.** SSO through OIDC to OpenBao or a cloud console, with group membership driving policy, so a leaver loses access when their SSO account is disabled.

A credential that is issued per request and expires in minutes needs no rotation procedure. This is the strongest available answer to chapter 5: the access set of each value is one consumer, for a short time.

9.4 The models compared

	Encrypted files (SOPS + age)	SOPS + KMS	Self-hosted service (OpenBao)	Cloud manager	SaaS manager
Source of truth	git	git	service	cloud	vendor
Who can read	Recipients, for ever	KMS policy, at decrypt time	Policy, per request	IAM	Vendor RBAC
Audit of use	No	Decrypts	Every request	Every request	Every request

	Encrypted files (SOPS + age)	SOPS + KMS	Self-hosted service (OpenBao)	Cloud manager	SaaS manager
Revocation	Rotate everything	Stop future decrypts; rotate	Revoke leases; dynamic credentials expire	Rotate; IAM	Rotate
Dynamic credentials	No	No	Yes	Limited (rotation functions)	Some products
Runtime dependency	None	KMS at deploy	Service at start and renewal	Cloud API	Vendor API
Operating cost	Lowest	Low	Highest	Low	Lowest
Secret zero for hosts	age identity	Cloud credential	AppRole or platform identity	Cloud credential unless in-cloud	Service token
Kubernetes path	Flux, or ESO via another store	Same	ESO, CSI, injector	ESO, CSI	ESO, vendor operator

No column wins. The right one depends on the number of people who must lose access over time, whether credentials you issue (databases) dominate, what identity the hosts already have, and who will operate a tier-0 service.

9.5 Questions for an agent

Ask: "For this proposal, state the storage, delivery and identity choices separately, and what secret zero is for a host, a CI job and a person." A good answer has three short, distinct answers and a named

secret zero for each. Warning signs: a product name in place of a design; "the service handles security"; an identity that is a long-lived token stored in the same place as the secrets it unlocks.

9.6 Exercise 9: choose under constraints

For each situation, choose a storage model and a delivery pattern, and give the one fact that most drives the choice.

1. A personal project: one VM, three Compose services, one person, a database and two third-party API keys.
2. A nonprofit: eight hosts, four volunteers who change every year, no budget for operating infrastructure, deployments from GitHub Actions.
3. A company fleet of 40 hosts in two data centres, a dozen PostgreSQL databases, an auditor asking who accessed production credentials, and an SRE team already on call.

Worked answer

10. Kubernetes later (optional)

Read this chapter if Kubernetes is a realistic destination; it needs chapters 6 and 9. The aim is not to design a cluster but to make today's choice one that carries forward.

10.1 What a Kubernetes Secret is

A Secret is an API object whose values are base64-encoded: an encoding, not encryption (the documentation describes a base64 credential in one of its examples as "obscured but not secret"). By default Secrets are "stored unencrypted in the API server's underlying data store (etcd)" (Secrets). Three properties surprise people used to files on a host:

- **Creating a pod reads a secret.** "Anyone who is authorized to create a Pod in a namespace can use that access to read any Secret in that namespace", including indirectly through a Deployment. `list` on Secrets also returns their contents (good practices). Namespaces and who may deploy into them are therefore

the access-control boundary.

- **Encryption at rest is opt-in** through an `EncryptionConfiguration` on the API server. `identity` (the default) does not encrypt; `aescbc` is rated weak; `secretbox` is strong but keeps the key on the control-plane host; `kms` v2 wraps data keys with an external KMS and is the recommended provider (encrypt data).
- **Delivery.** Mounted Secrets live on a tmpfs on nodes that run pods needing them. Volume mounts are updated eventually when the Secret changes; `subPath` mounts and environment variables are not, until the pod restarts. The rotation lessons of chapter 6 carry over directly.

10.2 Getting secrets from a store into pods

Tool	What it does	Status (October 2026)
External Secrets Operator	Syncs values from a store (OpenBao/Vault, cloud managers, many SaaS) into Secret objects	v2.13.0; CNCF Sandbox; releases paused August–September 2025 for lack of maintainers, then resumed (pause)
Sealed Secrets	You encrypt to a controller's public key; the ciphertext lives in git; the controller decrypts into Secrets	v0.40.0; Bitnami (Broadcom); stated unaffected by the 2025 Bitnami image changes (issue)
Flux with SOPS	Flux decrypts SOPS files from git during reconciliation	Part of Flux (guide)
Secrets Store CSI Driver	Mounts values from a store as a volume, optionally without a Secret object	v1.6.1; kubernetes-sigs
Agent injector	Adds an OpenBao or Vault Agent sidecar that renders files, as in lab 4	OpenBao Helm chart; vault-k8s 1.7.6

ESO's pause is a maintenance risk worth weighing: an operator that holds read access to every secret store is a high-value component, and its health depends on a volunteer maintainer team. ESO's provider for HashiCorp Vault is the usual route to OpenBao; check compatibility for the versions in use (provider).

10.3 How today's choice carries forward

- **SOPS + age today** carries over to Flux, which decrypts the same files, with the cluster holding an age identity. Recipient management and revocation-by-rotation stay the same.
- **OpenBao today** carries over through Kubernetes auth (pods authenticate with service-account tokens, removing AppRole secret zero) plus ESO, CSI or the injector. Policies and dynamic credentials stay the same.
- **A cloud manager today** carries over through ESO or the provider's CSI plug-in, with workload identity if the cluster runs in that cloud.

Delivery and identity change with the platform; storage need not. That is the practical reason to keep the three decisions separate.

10.4 Questions for an agent

Ask: "Who can read this Secret, counting everyone who can create pods or workloads in its namespace and everyone with `list`, and which encryption provider protects it in etcd?" A good answer lists roles and service accounts from the RBAC bindings, not just those with explicit `get` on Secrets, and names the provider from the API server's encryption configuration. Warning signs: "it's base64-encoded"; access described only by `get` permissions; encryption at rest assumed because the cluster is managed.

Ask: "When this Secret changes, how does each pod that uses it pick up the new value?" A good answer distinguishes volume mounts (eventually updated), `subPath` mounts and environment variables (only on restart), and says what triggers the restart. Warning sign: "the operator syncs it" offered as the whole answer, since syncing the Secret object is not the same as the process reading it.

10.5 Exercise 10: predict an update

A Deployment uses one Secret three ways: as an environment variable `API_KEY`, as a volume mounted at `/etc/creds/`, and as a single file mounted with `subPath` at `/etc/app/token`. ESO updates the Secret with a new value. Without a pod restart, which of the three show the new value, and what would an agent need to add so that the application uses it?

Worked answer

11. Choosing and migrating

11.1 Questions that change the answer

Answer these first; each one moves the choice between columns of the comparison table.

1. **Turnover.** How many people will need to lose access per year? Each departure costs a full rotation in model one.
2. **Issued credentials.** What fraction of secrets are database or internal service credentials you control? Those benefit most from dynamic credentials.
3. **Audit.** Does anyone (customer, auditor, regulator) need to know who *read* a production credential, not just who could?
4. **Identity.** What do hosts, CI and people already authenticate with? SSO, a cloud, a CI provider with OIDC, host certificates?
5. **Operations.** Is there a team that will be paged when the secret service is sealed at 03:00? If not, the service must be hosted.
6. **Platform horizon.** Will Kubernetes arrive, and on which cloud, if any?

11.2 Three candidate designs

Design 1: SOPS in the configuration repository. Production secrets move from the private repository into SOPS files with age recipients per person and per host (or a KMS key per environment); Ansible decrypts at deploy and writes files for Compose `file:` secrets (pattern A); CI uses its own recipient for non-production only. Cheapest and fastest, keeps the git workflow, no new runtime dependency. Weak on

revocation and audit; no dynamic credentials.

Design 2: self-hosted OpenBao. A three-node OpenBao cluster with integrated storage and KMS or Shamir unsealing; host agents (pattern B) or sidecars (pattern C); dynamic credentials for PostgreSQL; KV for third-party keys; SSO for people, JWT auth for CI, AppRole with constrained secret IDs for hosts until Kubernetes auth is available; audit to the log pipeline. SOPS remains for the small set of bootstrap and break-glass material: recovery keys, initial secret IDs, the KMS credential if any. Strongest properties, highest cost.

Design 3: hosted manager. A cloud manager where the workloads already live in that cloud, or a SaaS manager otherwise, with its CLI or agent writing files (pattern A or B). Low operating cost and good audit; a vendor dependency, a per-host service credential as secret zero outside a cloud, and (for SaaS) a vendor whose product lifetime is not under your control.

Recommendation, for a fleet of a few dozen Compose hosts with configuration management, Kubernetes about a year away, no cloud committed, and both hosted and self-hosted acceptable: adopt design 1 now, as a deliberate first stage, and plan design 2 as the target if two or more of these hold: frequent turnover, many issued database credentials, a requirement to audit reads, or a team able to run a tier-0 service. Design 1 removes the worst current exposures within weeks and does not block design 2: the SOPS files become the import source for OpenBao's KV store and shrink to bootstrap material. Choose design 3 instead of 2 when the fleet moves into one cloud, or when no one will operate OpenBao. This is a judgement under stated assumptions, not a universal answer.

11.3 A staged migration

Stage 0: stop the bleeding (days).

- Inventory: run a secret scanner such as gitleaks or TruffleHog over every repository *including full history*, over image histories and configurations, Compose files, `.env` files on hosts, CI variables and `~/.docker/config.json` on build hosts.
- Classify each finding by kind (chapter 5) and by consumer.
- Turn on `# check=error=true` in Dockerfiles and a scanner in CI

and pre-commit hooks, so new leaks fail early. Neither catches everything.
- Remove secrets from `ENV`, `ARG` and `-e`, using build secrets and Compose secrets (chapter 6).

Stage 1: new store, one service end to end. Choose a low-risk service with a database credential. Move its secrets into the new store, deliver them by the chosen pattern, and **rotate them** during the move: a value that was in the old repository is compromised by definition. Write down the procedure as you go; it becomes the runbook.

Stage 2: the rest, by blast radius. Highest-value and most widely copied secrets first. Each migration ends with a rotation.

Stage 3: retire the old repository. Archive it read-only for a defined period, then delete it. Neither step revokes anything; the rotations in stages 1–2 did that.

11.4 Rotation without an outage

The lab's failed login after "rotation" (chapter 6) has a standard fix: let two credentials be valid during the change.

1. Create the new credential at the authority while the old one still works (a second database user, a second API key, or a password change on a second account that consumers switch to).
2. Deliver it, and make every consumer pick it up (restart or reload).
3. Verify from the authority's side that no consumer still uses the old one (database session list, provider's key-usage log).
4. Revoke the old credential, and end its existing sessions.

Dynamic credentials do this continuously; static ones need it written down per secret. A rotation is complete only when step 3 has evidence.

11.5 Acceptance criteria for a migrated service

A migration proposal from an agent should come with criteria like these, each with the command or log that demonstrates it:

- The service starts from a clean host with only the documented bootstrap credential.

- No secret value appears in `docker inspect`, `docker history`, `docker compose config` output, the image layers, the CI log, or the repository's working tree and history since migration.
- Each secret file is readable by the container's user and by no other host user.
- Rotating each secret by the documented procedure changes what the service uses, without an outage, and the old value is rejected by the authority.
- Revoking the host's or service's identity stops it obtaining secrets; the failure is visible in monitoring.
- With the secret store unavailable, the documented degraded behaviour happens (keep running, fail to start, or similar).
- The audit record shows the service's own identity fetching its secrets and nothing else.

11.6 Leak response

When a secret is known or suspected to be exposed:

1. **Rotate first.** Evidence gathering does not reduce exposure; rotation does. Use the dual-credential procedure if there is time, a hard switch if not.
2. **Assess use.** Check the authority's logs (database connections, provider API logs, cloud audit logs) for use of the old value between exposure and revocation.
3. **Find the cause** and the access set: where the copy went and who could read it.
4. **Clean up last**, if at all. Rewriting git history or deleting an image tag reduces future discovery but does not recall copies already made.

An agent that proposes `git filter-repo` before rotation has the order wrong.

11.7 Diagnosis

Symptom	Likely causes	Evidence that distinguishes them
`Permission denied` on `/run/secrets/x`	Host file owner or mode versus container UID; `uid`/`mode` given on a `file:` source	`ls -ln` on host; `docker inspect -f '{{.Config.User}}'`; Compose warnings
Service uses old value after rotation	Single-file bind mount pinned the old inode; process read once at start; `up -d` did not recreate	Compare host file and `docker exec cat`; container start time; application reload logic
Authentication fails after rotation	Authority not updated (database still has old password); consumer not yet reloaded	Authority's log for which user and value failed; `ALTER ROLE` history
Agent renders nothing; "no known role ID"	Agent cannot read its credential files (directory mode, UID); wrong path	`docker exec` as the agent's UID, `ls -ln`; agent log
Intermittent failures at a fixed interval	Credential expiry (max TTL) with a pool holding connections, or renewal failing	Lease TTLs; times of failures versus issue times; agent log
Nothing can start; secrets unreachable	Service sealed after restart; network; expired agent token	`bao status` (`Sealed`); agent log; audit log
SOPS: "Failed to get the data key required to decrypt the SOPS file"	Identity not a recipient of that file; wrong `SOPS_AGE_KEY_FILE`	`.sops.yaml` rule for the path; recipients in the file's `sops` metadata
Secret appears in CI log	`docker compose config` or `set -x` on interpolated values	Search logs for the value; replace with `--no-interpolate` or file secrets

Ask for the evidence column before accepting an agent's diagnosis. "I

changed the mode and restarted, and it works now" is a fix without a cause; the cause determines whether the same failure is waiting on the other 29 hosts.

11.8 Questions for an agent

Ask: "What is the rollback if this migration fails halfway, and which values will already have been rotated?" A good answer recognises that rotation is one-way: once the old value is revoked, rolling back the delivery mechanism without the new value breaks the service. Warning signs: rollback described as "revert the commit"; no list of values rotated in each step.

11.9 Exercise 11: acceptance criteria

Write five acceptance criteria, each with the evidence that demonstrates it, for moving one Compose service's PostgreSQL password from a private repository to SOPS-encrypted files deployed by Ansible (design 1, pattern A).

Worked answer

11.10 Exercise 12: diagnose

After moving a service to OpenBao dynamic credentials with a host agent (pattern B, `default_ttl=1h`, `max_ttl=24h`; the agent renders the file but no `exec` is configured), the service logs `password authentication failed` for a few minutes roughly once a day, then recovers without intervention. The agent's log shows a new credential rendered shortly before each episode. An agent proposes raising `max_ttl` to 30 days. Give the most likely cause, the evidence that would confirm it, and a better fix.

Worked answer

Worked answers

Exercise 1

Back to exercise 1

1. Recoverable. `COPY` is its own instruction and produces a layer containing the key; the later `rm` only adds a whiteout. Had the key arrived and been removed within one `RUN` (or better, through `--mount=type=ssh`), no layer would contain it.
2. Not in the final image's layers, configuration or history: only the final stage becomes the image. It is still in the builder's cache and in the history of the `build` stage on the build host, and if the image is pushed with `mode=max` provenance the build argument's value is recorded in the attestation. Build arguments are the wrong channel even here.
3. Recoverable from the layer produced by that `RUN`: the mount kept the token out of the layer, then the script wrote it to `~/.npmrc`. Remove the file in the same `RUN`, or point the tool's configuration at the mounted secret (for npm, `NPM_CONFIG_USERCONFIG=/run/secrets/npmrc`) so that nothing is written.

Exercise 2

Back to exercise 2

In order of severity:

1. `ARG PIP_INDEX_URL` carries a token, and the argument is in scope for `RUN pip install`, so the full URL with credentials is recorded in that step's history entry: anyone with the image has the index credentials. Use `RUN --mount=type=secret,id=pip_index,env=PIP_INDEX_URL pip install -r requirements.txt` and build with `--secret id=pip_index,env=PIP_INDEX_URL`.
2. `COPY . /app` with no `.dockerignore` sends the whole context, which may include `.git` (with any history of secrets), `.env` files and keys, into a layer. Add a `.dockerignore` that excludes them, or copy only what the application needs.
3. `ENV SENTRY_DSN=…` bakes environment-specific configuration into the image. A Sentry DSN for event submission is designed to be

embedded in clients and is low sensitivity, but it ties the image to one environment and normalises `ENV` for credentials. Supply it at run time.

`docker build --check` catches **none** of these: `PIP_INDEX_URL` and `SENTRY_DSN` contain no word from the rule's list, and the rule does not look at `COPY` or values. This is why a clean check is weak evidence.

Exercise 3

Back to exercise 3

Observed on Compose 5.6.0 with this exact setup:

- `DATABASE_URL=postgres://api:s3cret@db/api`: interpolated from `.env` (a `DB_PASSWORD` exported in the shell running Compose would win).
- `LOG_LEVEL=info`: a literal in `environment:` beats `env_file` and the image's `ENV`. `.env`'s `debug` is used only for interpolation and nothing references it.
- `DB_PASSWORD=other`: from `api.env`. The `.env` value does not enter the container.

`docker compose config` printed `DATABASE_URL: postgres://api:s3cret@db/api` **and** `DB_PASSWORD: other`: it inlines `env_file` contents into `environment` as well as interpolating. Both passwords therefore appear in any log that runs it.

Exercise 4

Back to exercise 4

The access set includes at least: all five people, including the two leavers, through clones on laptops, laptop backups and any synced folders; the git host and its backups; the CI system, its runners' cached workspaces and anything that printed the value; every host running the container, where the value is in the container's configuration under `/var/lib/docker`, visible to `docker inspect` for root and `docker` group members, and in the process environment; the nightly host backups and everyone with access to backup storage; any error reports or logs that captured the environment. Removing the leavers' access changed none of this. The only way to make them lose access is to

rotate the password at the database (with a transition so the service stays up), and to stop delivering it through `-e` so that the next value has a smaller access set.

Exercise 5

Back to exercise 5

At start-up the worker, running as UID 1000, gets `Permission denied` reading `/run/secrets/smtp_password`: a `file:` secret is a bind mount, `uid` and `mode` are ignored for that source, and the host file is root-owned 0600. Compose prints a warning that `uid`/`mode` are unsupported outside Swarm, but `config` only validates and resolves the model; it never starts a container or reads the file as the container's user. Fixes:

- Have Ansible write the file owned by UID 1000 (or by a group the container user belongs to), mode 0400 or 0440, inside a root-owned 0700 directory. Correct and simple; couples the host's file ownership to the image's UID, which must be kept in step.
- Use an `environment:`-sourced secret, for which `uid`, `gid` and `mode` are honoured. Ownership is then set per container, but the value must be in the environment of the process running `docker compose`, and it is written into the container's writable layer.

Making the file 0444 inside a 0700 directory also works and was used in lab 2; it relies entirely on the directory to keep other host users out.

Exercise 6

Back to exercise 6

The directory mount lets the application *see* the new file, but it read `db.env` once, so it keeps the old credential in memory. Connections already open keep working even after revocation (observed in chapter 8: an open session survived the role being dropped). The failure appears later, when the pool opens a new connection (growth, a database restart, a recycled connection) with the old, now invalid credential, so it is intermittent and delayed. Fix: have the agent's `exec` reload or restart the application when the file changes, or make the application read the file whenever it opens a connection, and set the pool's max-

imum connection lifetime below the credential's lifetime. If revoked credentials must stop working immediately, the revocation statements must also terminate their sessions.

Exercise 7

Back to exercise 7

1. Remove Alice's recipient from both rules in `.sops.yaml`, run `sops updatekeys` on every file, commit. This closes *future* commits to her and nothing else.
2. `sops rotate` each file to issue new data keys. Hygiene; it does not affect history.
3. Rotate every credential in `prod/` and `dev/` at its authority, highest value first, using a transition where needed, and deploy the new values. This is the step that revokes; she has been able to decrypt every past commit for two years, and a wiped laptop does not prove the absence of other copies.
4. Revoke her other access (git host, SSO, hosts) and remove her identity from anywhere it was installed.
5. Verify from each authority that the old values are rejected.

With a KMS recipient used through SSO, disabling her SSO account stops all future decryptions, *including of old commits*, because every decryption needs a KMS call. Values she actually decrypted during the two years are still known to her, so they must still be rotated; the KMS's log of her decryption calls narrows the list (how precisely depends on whether the calls can be tied to files), instead of requiring every value to be assumed exposed.

Exercise 8

Back to exercise 8

A sound design:

- Ansible, authenticated to OpenBao as the operator (SSO) or as CI (JWT), creates the host's AppRole, or a role per service with a per-host secret ID, with a policy limited to that host's paths. The role ID is not secret on its own and can live in inventory.

- The secret ID is generated in pull mode, bound to the host's address (`secret_id_bound_cidrs`), and delivered as a response-wrapped token. Wrapping does not bind the token to the agent: whoever holds it can unwrap it, but only once, so an agent that finds its token already unwrapped has evidence of interception.
- The design must say what happens on every agent restart, host reboot or expiry of the agent's token. If the secret ID is single-use or short-lived and the token lives only in memory, each of those needs a fresh secret ID, so Ansible (or a boot-time hook that calls it) is on the start-up path. The alternative is a longer-lived, address-bound secret ID kept in a root-only file, which survives reboots but is then on the disk. That tradeoff between start-up dependency and material at rest is the decision to make explicitly.
- The agent keeps its token in memory and renews it; rendered secrets go to a tmpfs, not disk.
- Replacement is the same playbook, which creates a new secret ID and destroys the old one; it also runs when a host is rebuilt.

An attacker with one host's disk image gets whatever is on disk: the role ID, any persisted secret ID, and any rendered files not on tmpfs. With address binding, the secret ID cannot be used from elsewhere; without it, they can log in and read what that host's policy allows until the secret ID is destroyed. Per-host policies keep this to one host's secrets. Moving to platform identity (TLS certificates issued at provisioning, or Kubernetes auth later) removes the reusable secret ID.

Exercise 9

Back to exercise 9

1. SOPS with age (identities for the person and the VM), pattern A, or even root-only files on the host with Compose `file:` secrets. Driving fact: one person and one host; there is no one to revoke and nothing worth operating.
2. A hosted manager with SSO for volunteers and GitHub Actions OIDC for deployment, or SOPS with a KMS recipient tied to SSO if the organisation already has a cloud account. Driving facts: annual turnover (age recipients would mean a rotation campaign every year) and no operations budget (rules out OpenBao).

Either is defensible; the comparison should name its free-tier and lifetime risks.

3. A self-hosted OpenBao (or Vault) cluster with dynamic PostgreSQL credentials, audit to the log pipeline, and SSO and JWT auth. Driving facts: an auditor asking about reads (needs per-request audit) and many issued database credentials, with a team able to run a tier-0 service. A cloud manager is the alternative if the fleet moves into one cloud.

Exercise 10

Back to exercise 10

Only the volume mount at `/etc/creds/` changes, and only eventually, after the kubelet's next sync. The environment variable keeps the value from pod start, and the `subPath` file does not receive updates. Even the updated file helps only if the application re-reads it. To make the application use the new value, add a rollout on change (for example a checksum of the Secret in the pod template annotations, or a reloader that restarts the Deployment), or make the application read the mounted file when it needs the credential and stop using the environment variable and `subPath` mount for it.

Exercise 11

Back to exercise 11

For example:

1. The old password is rejected: `psql` with the old value fails, run after cut-over.
2. The value is not in plaintext in the repository: a scanner over history since migration finds nothing; `sops` metadata of the file lists exactly the intended recipients.
3. The container receives it as a file: `docker inspect` shows a read-only bind mount at `/run/secrets/…` and no password in `Config .Env`; `docker compose config` output contains no password.
4. Only the container user can read it on the host: `ls -ln` shows the intended owner and mode inside a 0700 directory; reading it as another unprivileged user fails.

5. Rotation works end to end: following the written procedure (`sops edit`, Ansible run, database `ALTER ROLE` with a transition) changes the password the service uses without failed requests in its logs, and criterion 1 then holds for the previous value.

Exercise 12

Back to exercise 12

Most likely: at the 24-hour `max_ttl` the lease cannot be renewed further, so the agent fetches a new credential and the old one expires. The application holds the old credential and keeps using it for new connections until something (a failed login, a periodic re-read, or a delayed restart) makes it reload, which explains a few minutes of failures and self-recovery. Confirm with PostgreSQL's log showing the *previous* user name failing, failures starting about 24 hours after that credential was issued, and the gap between the agent's render time and the application's reload. Raising `max_ttl` to 30 days only makes the outage monthly and turns short-lived credentials into long-lived ones. Better: reload or restart the application from the agent's `exec` when the file changes, or have it re-read the file for each new connection, and set the pool's connection lifetime below the credential's lifetime.

Field reference

Need	Command or setting
Image history with arguments	`docker history --no-trunc IMAGE`
Image configuration	`docker image inspect -f '{{json .Config}}' IMAGE`
Scan all layers	`docker save IMAGE` piped to `python3 scan_image.py VALUE` (lab 1)
Lint for secret-like names	`docker build --check .`; `# check=error=true`
Build secret from env / file	`--secret id=X,env=VAR`, `--secret id=X,src=PATH`

Need	Command or setting
Use build secret	`RUN --mount=type=secret,id=X[,env=VAR][,required=true]`
SSH during build	`RUN --mount=type=ssh` with `--ssh default`
Resolved Compose model	`docker compose config` (shows interpolated values); `--no-interpolate`
Container env and mounts	`docker inspect -f '{{json .Config.Env}}', '{{json .Mounts}}'`
Compose secret sources	`file:` (bind mount, host owner/mode); `environment:` (copied, `uid`/`gid`/`mode` honoured)
SOPS encrypt / decrypt	`sops encrypt -i F`; `sops decrypt F`; `--extract '["KEY"]'`
SOPS recipients / data key	`sops updatekeys -y F`; `sops rotate -i F`
SOPS into a process	`sops exec-env F CMD`; `sops exec-file F CMD`
age key pair	`age-keygen -o KEY`; `age-keygen -y KEY` prints the recipient
OpenBao state	`bao status` (`Sealed`, `Version`)
Issue / revoke	`bao read database/creds/ROLE`; `bao lease revoke -prefix database/creds/ROLE`
Rotate DB admin	`bao write -f database/rotate-root/NAME`
Audit device (OpenBao 2.3.2 and later)	`audit "file" "NAME" { options { file_path = … } }` in server config

Labs, sources and verification

Files

All under `docker-secrets/examples/`:

- `leaks/`: `Dockerfile.leaky`, `Dockerfile.fixed`, `scan_image.py`, `lab.sh`. Builds two images and runs one container; removes them.
- `compose/`: `compose.yaml`, `lab.sh`. Destroys and recreates the Compose project `secrets-lab`.
- `sops/`: `lab.sh`. Downloads SOPS 3.13.3 and age 1.3.2 into `sops/.bin` after checking their SHA-256 digests, and works in `sops/work`.
- `openbao/`: `compose.yaml`, `agent.hcl`, `server.hcl`, `lab.sh`. Destroys and recreates the Compose project `openbao-lab`; takes about two minutes.

Every value in the labs is a lab marker. Run them only against a disposable Docker daemon; `README.md` shows how to use a Docker-in-Docker container for that.

Versions and sources

Target: Docker Engine 29.9 (29.9.2 current), Compose v5.6, Buildx 0.38, BuildKit 0.34, Dockerfile frontend 1.28; SOPS 3.13.3; age 1.3.2; OpenBao 2.7.1; PostgreSQL 18 (`postgres:18-alpine`); Alpine 3.24. Discussed, not run: Kubernetes 1.37, Vault 2.2, External Secrets Operator 2.13, Sealed Secrets 0.40, cloud and SaaS managers. Research date: 21 October 2026. Primary sources are linked beside each claim; documentation URLs are mutable and were read on that date. Chapter 1 was researched on 3 November 2026.

What was and was not verified

`verification.md` records the runs. Chapter 1's examples ran on 3 November 2026 on Docker Engine 29.9.2 with Compose 5.6.1, apart from those it lists. All four labs ran in full on Docker Engine 29.9.1 with Compose 5.6.0 and Buildx 0.38.0 (containerd image store), in a Docker-in-Docker container, plus SOPS 3.13.3 and age 1.3.2 on the

host; exercise 3 and the open-session revocation check were run separately on the same daemon. Not run: Swarm secrets, rootless mode, attestations, multi-platform builds, Bake, Kubernetes and every tool in chapter 10, Vault, cloud and SaaS managers, an OpenBao deployment outside dev mode (storage, HA, unseal, response wrapping, `exec` restarts), Ansible's `community.sops`, Flux, and the scanners named in chapter 11. Prices and product status come from vendor pages on the research date.